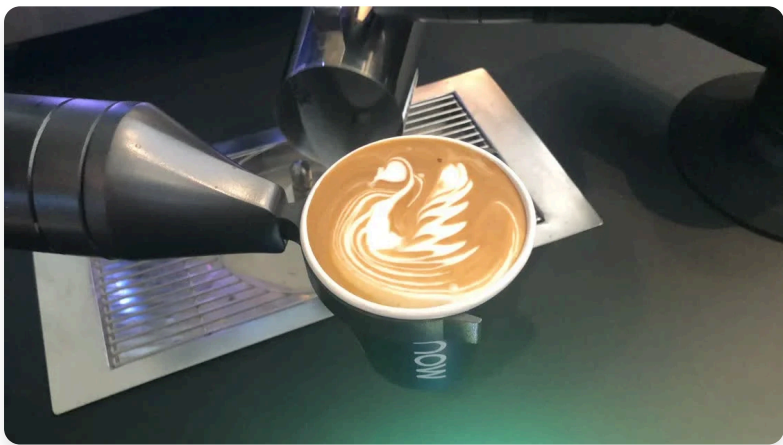


Auto Painter: From RGB Image to Pen Plot Trajectories

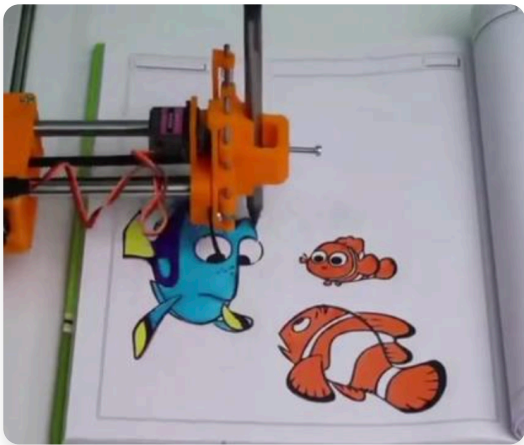
1. 背景

具身智能的快速发展让机器人从数字世界走向物理世界，机器学习模型被部署到真实的机械臂与移动平台上，实现感知—决策—控制的闭环。在咖啡拉花机器人中，轨迹规划算法让机械臂稳定控制奶泡与咖啡流；自动画画机器人则将图像生成与笔迹规划结合，把像素转化为可执行的笔画序列。这些前沿应用展示了视觉理解与运动控制的融合潜力，也为本项目的“Auto Painter”——从RGB图像到可执行的笔绘轨迹——提供了现实场景与技术参考。

咖啡拉花机器人



自动画画机器人



2. 任务定义

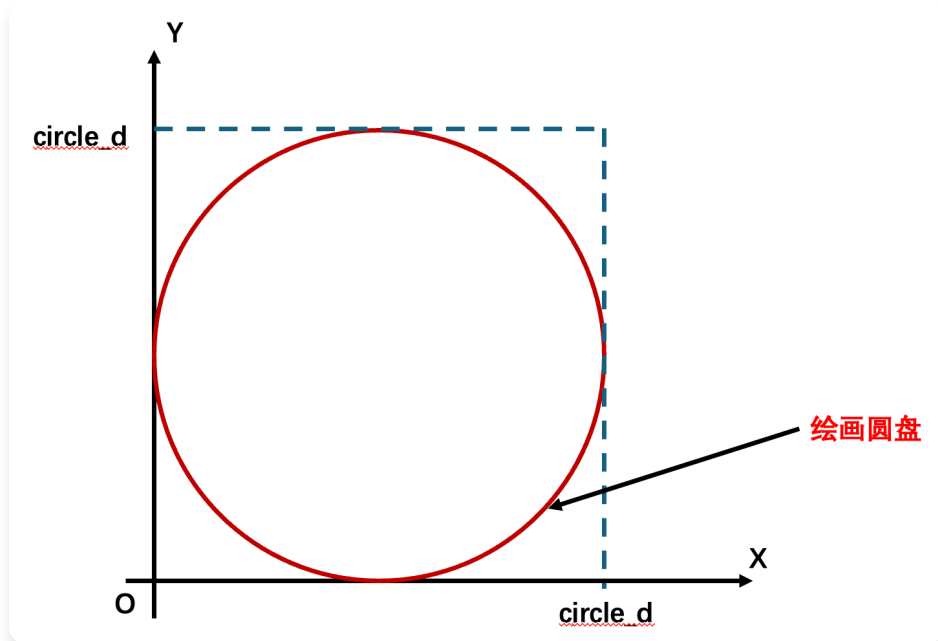
在Auto Painter项目中，你的任务是设计路径规划算法，为你的绘画机器人规划绘画轨迹。这一章节给出一些具体的定义，帮助你更好地理解任务。

2.1. 实体抽象

绘画机器人可以抽象成若干支画笔+一个绘画圆盘。机器人的画笔可以移动，移动精度为0.1mm。

- 【坐标系和长度单位】

经过工程师的精心调试，图像的一个像素已标定为物理世界中0.1mm * 0.1mm的格点，因此所有的坐标都是整数点，单位为1像素宽。控制指令中，坐标点（x，y）所处的坐标系和圆盘位置如图所示：



- 画笔：由笔头直径和画笔颜色来表征
 - 笔头直径 `pen_d` : int, 每次落笔会以画笔坐标为圆心形成一个直径为d的实心圆点。绘画机器人的笔头直径固定1mm。
 - 画笔颜色 `pen_color` : str, 画笔落下形成的实心圆点的颜色用RGB的十六进制字符串表示，比如红色的RGB是(255,0,0)，对应的字符串为"FF0000"。每种颜色的16进制字符串可以在[这里](#)查到
- 圆盘：圆盘是一个白色的圆形区域，由圆盘直径表征：
 - 圆盘直径 `circle_d` : int, 表示圆盘区域的直径，画笔的落点必须位于这个圆盘内部。绘画机器人的圆盘大小固定8.5cm。

2.2. 控制指令

你可以给机器人发送指令，让机器人完成绘画动作，不同指令之间以空格分隔。你可以发送的指令包括：

- **落笔指令**："8FDD 0000"。该指令让机器人把选中的笔头落到圆盘上，形成一个笔画圆点。已经落笔时无法再次执行落笔指令。
- **抬笔指令**："8FEE 0000"。改指令让机器人抬起笔头，脱离圆盘。已经抬笔时无法再次执行抬笔命令。机器画笔的初始状态为抬笔状态。
- **移动指令**："XXXX, YYYY"。发送两个数的四位十六进制表示，分别表示x坐标和y坐标，将笔头移动到 (x, y) 点。如果是落笔状态移动，那么圆形笔头会沿途画出笔画轨迹，如果是抬笔状态移动，那么不会画出任何轨迹。机器人画笔初始位于 (0, 0) 点。
- **切换画笔指令**："<画笔代号> 0000"。通过发送四位笔头代号和0000分割符，让机器人拿起目标画笔。机器人初始处于未拿笔状态。在未拿笔状态无法执行落笔命令。在落笔状态无法执行切换画笔指令。目前机器人支持7种颜色的画笔，画笔代号和颜色的对应关系如下：

```
"8FF5": "000000" // 黑色
"8FF4": "0033CC" // 蓝色
```

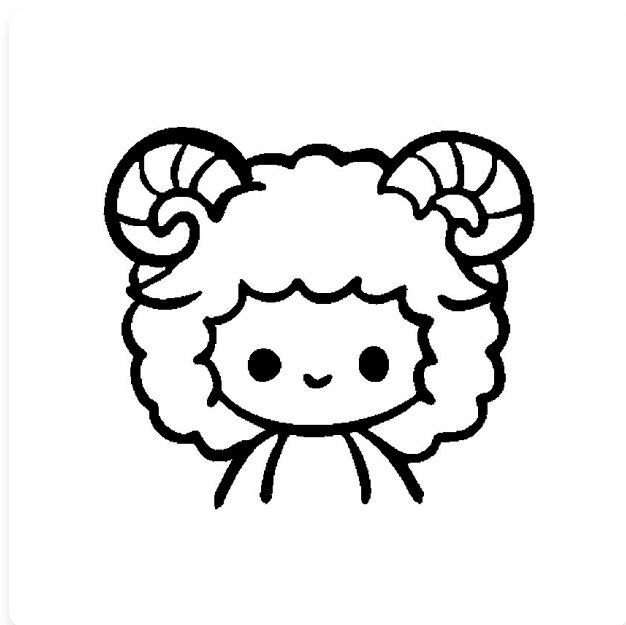
```
"8FF6": "FF9933" // 橙色
"8FF1": "FF0000" // 红色
"8FF3": "FFFF00" // 黄色
"8FF2": "00FF00" // 绿色
"8FF7": "00EEFF" // 青色
```

2.3. 输入输出与坐标系

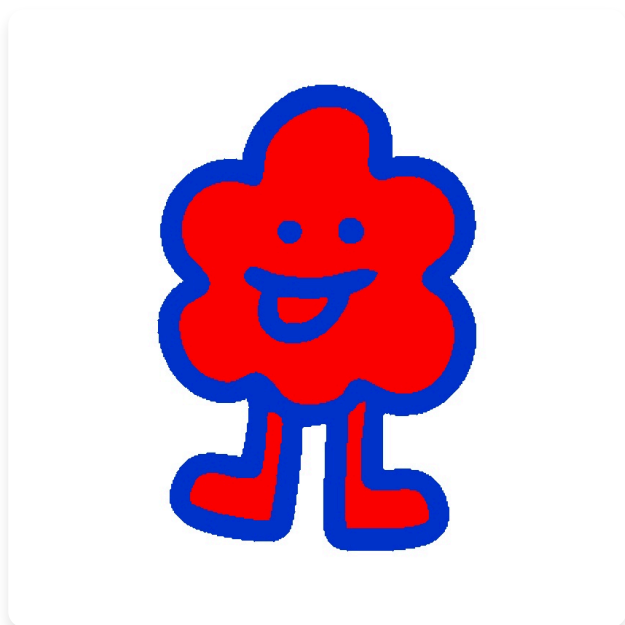
- 【输入】

Auto Painter算法的输入是用户上传的一张RGB颜色格式的JPG图像。该图像尺寸固定为850*850。图像背景为纯白色，前景为用户自定义的简笔画，可能仅有轮廓，也可能有色彩填充。前景保证出现在图像的内切圆中。

轮廓图像示例



有填充图像示例



- 【输出】

一份控制指令字符串，包含一连串空格分隔的[控制指令](#)。使得机器人按照指令文件进行操作即可画出用户上传的图像。下方是一个实例控制指令文件：首先拿上黑色画笔，移动到第一个坐标点(258, 515)，落笔，再水平移动到 (258, 516)，抬笔。

```
8FF5 0000 0102 0203 8FDD 0000 0102 0204 8FEE 0000
```

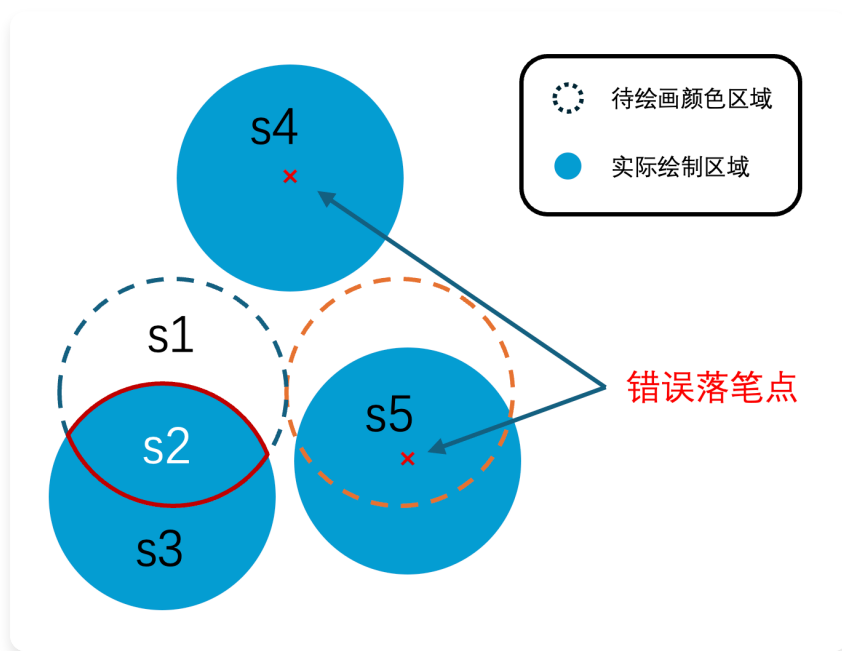
3. 评价指标

3.1. 精准度

机器人的绘画一定要最大程度逼近用户上传的图片。为了绘画结果的好坏，我们设计了下列评价指标：

1. **精确率 p** ：机器人画笔覆盖的像素中，处于正确区域的像素比例。
2. **召回率 r** ：所有正确区域中，被画笔正确覆盖的像素比例。
3. **错误数 N_w** ：完全落在不正确区域中的落笔点个数。一个落笔点只有其产生的覆盖范围不包含任何一个正确颜色像素，才会被认作错误点。

具体定义参考下图，虚线框为待绘画颜色区域，实心框为实际绘制区域。

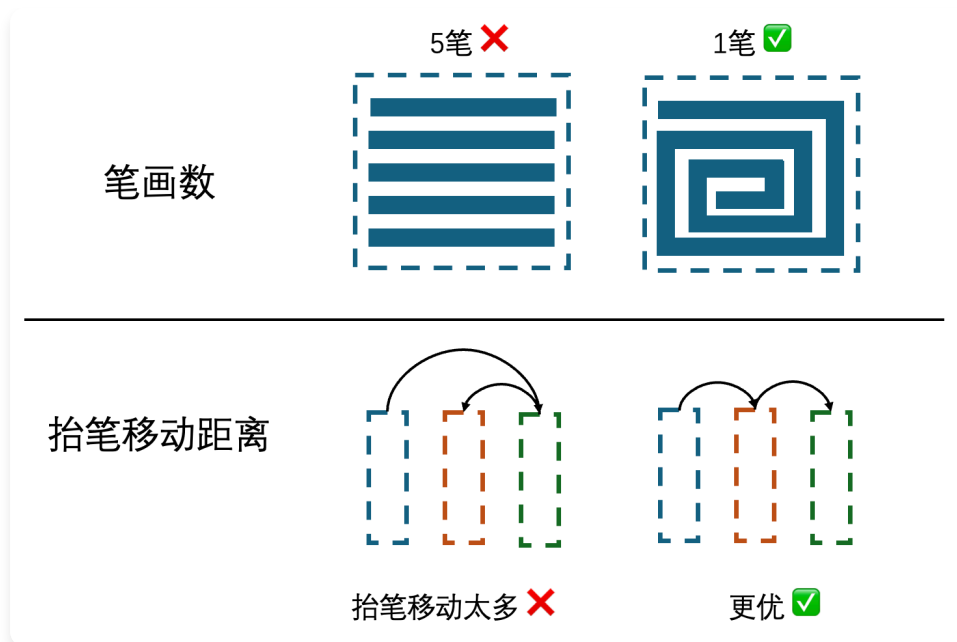


此图中， $p = \frac{s_2}{s_2+s_3+s_4+s_5}$, $r = \frac{s_2}{s_1+s_2}$, $N_w = 2$.

3.2. 绘画效率

为了节省成本，我们希望用尽量少的颜料和时间完成绘制，具体来说我们要优化下列指标：

1. **抬笔次数 N_l** ：由于z轴电机速率有限，机器人抬笔落笔速度很慢。为了节省用户等待的时间，整幅图案需要用尽量少的笔画画完。
2. **空笔距离 d_u** ：为了提高时间的利用率，我们要尽量减少机器抬笔状态下移动的总距离。
3. **绘画距离 d_p** ：最后，我们认为机械笔头在落笔状态下每单位移动距离喷涂颜料量是一定的，所以为了节约颜料，我们还要尽量减少机器落笔状态下移动的总距离。



机器抬笔需要3s，空笔运动0.1mm需要0.005s，时间成本均为0.01元/千秒；落笔运动0.1mm需要0.01s，时间成本为0.03元/千秒。所以一个绘画方案的总开销 c 是

$$c = N_l * 3 * 0.01 + d_u * 0.005 * 0.01 + d_p * 0.01 * 0.03$$

3.3. 评价分数

我们不能忍受错误点，所以一个错误点会直接导致评价分数-1。同时评价分数也会随绘画的开销线性减少。总的来看，一个机器人绘画方案的评价得分如下式：

$$\text{EvaluationScore} = 100 * \left(\frac{2}{\frac{1}{p} + \frac{1}{r}} - 0.01N_w - 0.001c \right)$$

4. 提交与计分

4.1. 评测网站

我们开发了一个[Auto Painter评测网站](#)用于本项目的自动评测和分数统计。同学们可以使用学号登录，登录之后才能进入提交页面。每个同学的初始密码为学号后六位，请及时修改密码。有任何问题请及时联系助教。

4.2. 帮助文档

在网站的“文档”页面包含您可能会用到的资料和文档，助教会陆续上传，保持关注！

4.3. 检查点

为了确保同学们顺利完成大作业，我们设置了三个检查点。

1. **安装OpenCV:** 同学们首先需要在自己的电脑上从源码编译OpenCV，并在编译前开启“BUILD_EXAMPLES”编译选项。这样编译器会在 build/bin/ 目录下编译出一系列OpenCV的

演示代码。编译成功后，您可以通过回答网站“提交”页面上“检查点一”中的问题来获取检查点一的分数。

2. **理解题目含义**: 同学们需要仔细阅读本作业要求，理解题目含义，然后去回答“检查点二”的问题获取检查点分数。
3. **上传诚信证明**: 每次提交算法结果之后，您需要提交一份诚信证明。在本次作业中，我们要求该证明为您开发算法的git历史。请按照[诚信审查](#)中的指导及时上传，上传后可以获得检查点分数。

4.4. 评测数据

通过点击网站上方的“下载数据集”按钮，大家可以下载评测数据集压缩包，用于开发和调试。数据集包含11张JPEG格式的图片，包含不同颜色的轮廓型、填充型简笔画。

4.5. 上传评测文件

算法开发完毕后，请为评测数据集中所有图片生成控制指令，生成结果请组织成一份JSON格式的评测文件（一定是.json后缀），文件格式如下：

```
// 此文件名为submission.json
{
  "results": {
    "<图片文件名1>": "控制指令字符串1",
    "<图片文件名2>": "控制指令字符串2",
    ...
  }
}
```

注意“results”中的图片文件名必须和数据集中的图片文件名完全一样，包括后缀名。

生成json文件后，你需要登录网站，前往“提交”页面，在“最终任务：提交文件”中提交你的json文件。每次提交评测会导致您上一份诚信声明失效，记得重新提交最新的诚信声明。

4.6. 计分规则

在你的“提交”界面，你会看到一个里程碑式的进度条，实时显示你的分数和Top5同学的分数情况（学号只展示后四位）。

您的分数计算规则如下：

1. 完成每个检查点获得20分。三个检查点共60分。
2. 提交评测的评测结果占40分。助教在这项任务中按照[评价指标](#)里的算法拿到了约83分的评价分数，代码量大约在500行。我们以此分数为95分线，您的评测得分算法为

$$\text{评测分数} = \min(\text{EvaluationScore}/83.04 * 0.95, 1.0) * 40.$$

3. 总得分为

总分 = 检查点分数 + 评测分数.

4.7. 诚信审查

1. 请设置并保管好评测网站上的账户密码，防止他人冒名提交。
2. 请在开发过程中使用git管理你的代码，在截止日期前，请导出你本地的git提交历史，保存在log.txt 文件中，于网站的“诚实审查”页面提交。我们会人工审核提交历史，对于疑似异常的历史，我们会安排同学进行线下code review。

```
git log --patch --pretty=format:"%h - %an, %ad : %s" --date=iso > log.txt
```

4.8. 网络安全

本评测服务部署在私人(土豆)服务器，为了保证教学活动正常进行，严禁任何形式的网络攻击行为，尤其是DDoS攻击。开发者保留根据后台日志追究攻击者责任的权利。